

Inventory Project Status and Roadmap

Completed features, missing/risky items, upcoming features, and best next steps

Project	Production-Ready Containerized Inventory & Order Management System
Stack	React, FastAPI, PostgreSQL, Docker, Docker Compose, Git
Purpose	First-round software engineer technical assessment and interview preparation

How to use this document

Use this document as a practical checklist before submission and as talking points during the technical interview. It focuses on what is complete, what still needs attention, and what can be added after submission without creating unnecessary risk.

1. Completed Features

The current project already covers the core scope expected in the first-round assessment. It is not just a basic CRUD demo; it includes backend business rules, frontend state management, authentication, Docker setup, testing, and deployment preparation.

Area	Completed implementation	Why it matters
Architecture	React frontend, Python FastAPI backend, PostgreSQL database, Docker Compose, and Git-based version control are present.	Shows correct full-stack structure and assessment alignment.
Backend API	FastAPI app has separated routers for auth, products, customers, orders, and dashboard.	Clean modular backend design.
Product management	Create, list, get by ID, update, delete, SKU uniqueness, price and quantity validation, category and description support.	Meets and exceeds product requirement.
Customer management	Create, list, get by ID, update, delete, email uniqueness, phone and email validation, address support.	Meets and exceeds customer requirement.
Order management	Create orders with customer and product references, list orders, get order details, update status, cancel/delete order.	Meets order requirement and gives realistic order lifecycle.
Inventory logic	Orders fail when stock is insufficient; stock reduces automatically on order creation; cancelled/deleted orders restore stock.	This is one of the most important assessment rules.
Dashboard	Summary includes total products, customers, orders, low stock, pending/fulfilled/cancelled orders, inventory value, and sales amount.	Good reviewer-facing feature.
Authentication	JWT login/register flow, password hashing, protected API routes, token storage on frontend.	Adds production-style access control beyond base requirement.
Frontend UX	Responsive dashboard, product manager, customer manager, order manager, messages, pagination, status filters, order details modal.	Makes the app demonstrable to reviewers.
Testing	Pytest covers product, customer, order, dashboard, health, stock reduction, insufficient inventory, duplicate SKU/email, and order detail cases.	Gives confidence and interview talking points.
Docker	Backend Dockerfile, frontend Dockerfile, Docker Compose, PostgreSQL service, named DB volume, .dockerignore files.	Mandatory requirement covered.
Deployment prep	Render config, Vercel-compatible frontend, environment-based API URL, README deployment guidance.	Ready for free online hosting after final URLs are added.

Completed Backend Details

- Products: backend validates product name, SKU, price, quantity, and optional category/description. SKU is normalized to uppercase so duplicate values are handled consistently.
- Customers: backend validates email format and stores normalized lowercase email. This helps avoid duplicate customer records such as TEST@EMAIL.COM and test@email.com.
- Orders: backend calculates total amount from product prices instead of trusting frontend values. This is correct because money and stock rules must remain server-side.
- Inventory: backend checks requested quantity against available quantity. If stock is insufficient, it returns an error and does not create the order.
- Order status: pending, fulfilled, and cancelled states make the order workflow more realistic than only create/delete.
- Dashboard: backend aggregates important business metrics, which gives the frontend a real summary screen rather than only CRUD pages.

- Authentication: APIs are protected with JWT, so products, customers, orders, and dashboard are not openly accessible without login.

Completed Frontend Details

- Auth page provides sign-in and create-account flow and saves login session using local storage.
- Central API wrapper attaches Bearer token automatically and redirects to login on 401 responses.
- Dashboard displays summary cards and order/inventory metrics useful for demo.
- Product manager supports add, edit, delete, search, category filter, validation, and pagination.
- Customer manager supports add, edit, delete, search, validation, and pagination.
- Order manager supports customer/product selection, multiple order rows, estimated total, notes, status filter, fulfill/cancel/reactivate actions, and view details modal.
- Message component provides clear success/error feedback after operations.

Assessment Requirement Coverage Matrix

Requirement	Status	Notes
React frontend	Completed	React + Vite application with component-based structure.
Python API	Completed	FastAPI backend with REST endpoints.
PostgreSQL	Completed	PostgreSQL service in Docker Compose and SQLAlchemy ORM.
Dockerfile backend	Completed	Backend container build is included.
Dockerfile frontend	Completed	Frontend build served with Nginx.
Docker Compose	Completed	Runs frontend, backend, and DB together.
Unique SKU	Completed	Database uniqueness + backend conflict handling.
Unique email	Completed	Database uniqueness + backend conflict handling.
Insufficient stock blocked	Completed	Order creation checks stock first.
Stock auto-reduction	Completed	Product quantity reduces on successful order.
Total amount backend-calculated	Completed	Total calculated using product price and quantity.
Live frontend URL	Pending	Deploy on Vercel and add URL.
Live backend URL	Pending	Deploy on Render and add URL.
Docker Hub backend image	Pending	Build and push backend image.

2. Missing or Risky Items

These are not major feature gaps, but they are the items most likely to create problems during review, deployment, or interview. Fix these before final submission.

No.	Risk/Missing item	Priority	Recommended action
1	Live deployment URLs not added	High	Deploy backend and frontend, then update README and assessment form with exact URLs.
2	Docker Hub backend image pending	High	Build backend image, push to Docker Hub, and include public image link.
3	README placeholders	High	Replace sample repository, Docker, Render, and Vercel placeholders with real links.
4	SECRET_KEY in deployment	High	Set SECRET_KEY in Render environment variables. Do not rely on local defaults.
5	.env exposure risk	High	Commit only .env.example. Never commit real .env with DB password or secret key.
6	test.db untracked file	Medium	Delete backend/test.db after pytest. Add *.db to .gitignore.
7	Database migrations basic	Medium	For assessment create_all is acceptable, but Alembic should be used for production schema changes.
8	Pydantic V2 warning	Low	Replace class Config with ConfigDict later. Not a blocker.
9	No screenshots in README	Medium	Add screenshots of dashboard, product list, order modal, API docs, and Docker running.
10	No final demo credentials section	Medium	Add demo login details after seeding data so reviewer can test quickly.

Commands to Keep the Repository Clean

Before every commit, run these commands from the project root:

```
git status
Remove-Item backend\test.db -ErrorAction SilentlyContinue
git status
```

If backend/test.db appears again after tests, delete it again. Test databases and real .env files must not be committed.

Deployment Risk Checklist

- Render backend must use the cloud DATABASE_URL, not localhost and not db unless running inside Docker Compose.
- Vercel frontend must set VITE_API_URL to the Render backend URL.
- Backend CORS_ORIGINS must include the final Vercel frontend URL.
- Docker Compose should use db:5432 internally; local uvicorn should use localhost:5432 or localhost:5433 depending on your mapped DB port.
- Do not submit until /health, /health/db, /docs, login, create product, create customer, and create order work online.

3. Upcoming Features After Submission

These features can make the system stronger in future versions. They are good interview discussion points, but they should not delay the first submission unless already implemented and tested.

Feature	Description	Why useful
Role-Based Access Control	Admin can create/update/delete while viewer can only read data.	High interview value because it shows real authorization design.
Inventory Ledger	Every stock movement is stored as an audit record: order created, order cancelled, manual adjustment.	Makes inventory tracking production-grade.
Supplier Management	Add supplier table and map products to suppliers.	Extends project from sales order system to procurement flow.
Purchase Orders	Create purchase orders to restock low-inventory products.	Natural next module after low-stock dashboard.
Audit Logs	Track user, action, entity, timestamp, old value, and new value.	Useful for accountability and debugging.
CSV/PDF Export	Export products, customers, low stock list, and order reports.	Reviewer-friendly and business-friendly feature.
Date Range Filters	Filter orders by date, month, customer, and status.	Improves order search and reporting.
Email Alerts	Send notification when stock goes below threshold.	Good automation feature.
Alembic Migrations	Manage DB schema changes safely across environments.	Important production improvement.
Refresh Tokens	Improve login security with short access tokens and refresh token rotation.	Good security interview topic.
CI/CD Pipeline	GitHub Actions to run tests on every push.	Shows engineering maturity.
Monitoring and Logs	Structured logs, request IDs, health checks, error tracking.	Production readiness feature.

Suggested Future Database Tables

Table	Fields	Purpose
stock_movements	id, product_id, movement_type, quantity, reason, created_by, created_at	Track every inventory change.
suppliers	id, name, email, phone, address, created_at	Supplier management.
purchase_orders	id, supplier_id, status, total_amount, created_at	Restocking workflow.
audit_logs	id, user_id, action, entity_type, entity_id, old_data, new_data, created_at	Action history.

4. Best Next Step Now

At this stage, the best next step is not another large feature. The best next step is to finish verification, commit cleanly, deploy, and submit.

Step	Action	Command/Place	Expected outcome
Step 1	Run backend tests with venv Python	<code>.\venv\Scripts\python.exe -m pytest app\tests -v</code>	Expected: 30 passed.
Step 2	Delete test DB	<code>Remove-Item backend\test.db -ErrorAction SilentlyContinue</code>	Prevents accidental commit of local DB file.
Step 3	Build frontend	<code>npm run build</code>	Confirms React code compiles for deployment.
Step 4	Commit order details modal	<code>git add ... && git commit -m "Add order details modal"</code>	Saves final visible feature.
Step 5	Run Docker Compose	<code>docker compose up --build</code>	Confirms full local container setup.
Step 6	Push backend Docker image	<code>docker build / docker push</code>	Required assessment deliverable.
Step 7	Deploy backend	Render web service	Get live backend API URL.
Step 8	Deploy frontend	Vercel project	Get live frontend URL.
Step 9	Update README	Add GitHub, Docker Hub, live frontend, live backend, demo login	Makes submission professional.
Step 10	Submit assessment form	Paste final links	Complete first-round submission.

Final Advice

- Do not add risky large features before deployment. A stable, deployed project is stronger than an unfinished advanced feature.
- During interview, emphasize business rules, backend validation, transactions, Docker Compose, deployment, tests, and your approach to production readiness.
- Keep a short demo flow ready: login, dashboard, add product, add customer, create order, show stock reduction, open order details, cancel order, show stock restore.